

Dmitri Lvovich

Montreal, Canada | +1 (819) 500-0776 | dmitri.lvovich@yahoo.ca | [Portfolio](#) | [GitHub](#) | [LinkedIn](#)

EDUCATION

McGill University

Bachelor of Science, Major in Mathematics and Computer Science

GPA: **3.72/4.00**

Montreal, Canada

Aug 2023 – Apr 2027

Relevant Coursework: Advanced Problem Solving in Competitive Programming, Algorithmic Game Theory, Distributed Systems, Advanced Algorithms, Applied Machine Learning

WORK EXPERIENCE

Amazon – AWS Lambda

Software Engineer Intern

Vancouver, Canada

May 2025 – Jul 2025

- Designed and prototyped an open-source Apache Flink stream-processing solution for AWS Lambda's Event Source Mapping team, targeting high-throughput, large-scale event workloads
- Demonstrated a **20.3% increase** in throughput compared to the existing architecture

Transport Canada

Software Developer Intern

Ottawa, Canada

May 2024 – Aug 2024

- Improved an office booking app used by 5000+ employees by resolving scheduling issues
- Enhanced internal tools with new features: Excel export-to-email functionality, notification system for permission group modifications, bulk permission assignment automation, and double-booking resolution plugin

CONTESTS & AWARDS

- **13th in Canada** in Round 2 of Meta Hacker Cup 2025
- **School Champion:** 2023 Euclid Mathematics Contest; 2022 Canadian Senior Mathematics Contest

RESEARCH & PROJECTS

[Minecraft Structure Reconstruction](#) | *Research Project, Ongoing*

Developing a system that recovers the 3D block-level geometry of Minecraft structures from 2D input images

Distributed Travel Reservation System | *Java, TCP, RMI*

- Built a distributed, concurrent multi-server reservation system using Java RMI and TCP sockets
- Used targeted synchronization to prevent race conditions and overselling across shared resources

Operating Systems Shell & Scheduler | *C*

- Built a Unix-like shell supporting built-in commands, command chaining, and fork/exec/wait
- Implemented a multi-process scheduler (FCFS, SJF, Round-Robin, Aging) using custom PCBs and ready queues
- Implemented virtual memory management with demand paging and an LRU page replacement policy

MiniCaml Interpreter | *OCaml*

Implemented an interpreter for a functional language with capture-avoiding substitution, big-step evaluation, and type inference using unification

SKILLS

Competitive programming | C++ | C | Java | Python | OCaml | AWS